

Take-home project

OpenArm 2.0 — Data Collection Pipeline

Estimated time

1-2 hours (AI tools encouraged)

Format

GitHub repo + write-up

CONTEXT

You are building a data collection platform for the **OpenArm 2.0** robotic arm (docs: *docs.openarm.dev*). The arm communicates over **CAN FD at 5 Mbps** via *can0* and *can1* interfaces, and is equipped with 4 cameras: two wrist Arducams (left/right), one ceiling Arducam, and one ZED stereo head camera. The system is used to collect teleoperation demonstrations for robot learning.

THE TASK

Complete as many tasks as you can — you don't need to finish all of them. This project intentionally covers a wide range of areas: we want to understand your skillset, not test whether you know everything.

1. CAN interface setup

Follow the setup guide at *docs.openarm.dev* to configure the CAN FD interfaces. Run `openarm-can-cli can_configure` and set the zero position on both *can0* and *can1*. Show a screenshot or terminal output confirming the interfaces are UP and zero position is set.

2. CAN data reading

Read live joint position, velocity, and torque from the arm over CAN FD. If you don't have hardware access, mock the CAN data stream in software and note that clearly.

3. Multi-camera synchronization

Capture (or simulate) synchronized frames from all 4 cameras (wrist left/right, ceiling, ZED stereo). Describe how you handle frame timestamps and alignment with joint state data. How do you deal with cameras running at different frame rates?

4. Data storage backend

Store episodes (joint states + camera frames) in a structured format. Choose a storage format and justify your choice (e.g. HDF5, MCAP, zarr, custom). Include a simple REST API to list episodes, retrieve metadata, and download an episode.

5. Monitoring dashboard

A simple web dashboard showing live (or simulated) joint states, a camera feed preview, episode count, and a Start / Stop recording button. Any frontend framework is fine.

DELIVERABLES

GitHub repository with code for the tasks you completed, plus a **write-up or README** covering: which tasks you completed, your architecture and design decisions, key trade-offs you made, and what you would do next given more time or hardware access.

WHAT WE LOOK FOR

We evaluate based on what you submitted, not what you skipped. A candidate who completes tasks 1-2 with hardware and explains the rest clearly is equally strong as one who completes tasks 3-5 in software.

Hardware / embedded — CAN setup correctness, data parsing, real-time awareness.

Data pipeline — Sync logic, timestamping strategy, storage format rationale.

Backend — API design quality, code clarity, error handling.

Frontend / UI — Usability, live feedback, recording controls.

Write-up — Clarity of reasoning, trade-offs, honesty about what's missing.

NOTES

- AI tools (Cursor, Claude, ChatGPT, etc.) are **encouraged** — we care about the decisions you make and your ability to explain them, not whether you typed every line.
 - There is no single correct answer. We want to see how you think through system design under real-world constraints.
 - If you make simplifying assumptions, state them clearly in your write-up.
 - Questions? Reach out to your point of contact at DeepAware AI.
-